

WEEK – 5

BA (Business Analytics)

Dynamic Portfolio Risk Arbitrage & Value-at-Risk (VaR) Monte Carlo Simulator

1. Aim

To design a multi-asset financial risk simulator that estimates portfolio risk using historical covariance, Cholesky decomposition, Geometric Brownian Motion (GBM), Monte Carlo simulation, and 99% Value-at-Risk (VaR).

2. Objectives

- Compute the historical covariance matrix for 500 assets.
- Apply Cholesky decomposition to the covariance matrix to generate correlated random variables.
- Simulate multi-asset price paths using Geometric Brownian Motion.
- Estimate the distribution of future portfolio values using Monte Carlo simulation.
- Calculate the portfolio's 99% Value-at-Risk (VaR).

3. Methodology

Step	Process
1	Collect historical prices/returns for 500 assets.
2	Calculate daily logarithmic or percentage returns.
3	Compute the 500×500 historical covariance matrix.
4	Perform Cholesky decomposition: $\Sigma = LL^T$.
5	Generate independent standard normal random vectors Z .
6	Create correlated noise using $\varepsilon = LZ$.
7	Simulate asset prices using GBM.
8	Calculate portfolio values and profit/loss for each simulation.
9	Find the 1st percentile of the P&L distribution.
10	Report 99% VaR as the corresponding potential loss.

4. Mathematical Model

Historical covariance: For asset returns R , the covariance between assets i and j is:

$$\text{Cov}(R_i, R_j) = E[(R_i - \mu_i)(R_j - \mu_j)]$$

For 500 assets, the result is a 500×500 covariance matrix Σ .

Cholesky decomposition:

$$\Sigma = LL^T$$

where L is a lower-triangular matrix. If Z is a vector of independent standard normal variables, correlated shocks are generated as $\varepsilon = LZ$.

Geometric Brownian Motion:

$$S(t+\Delta t) = S(t) \times \exp[(\mu - \frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t} Z]$$

For the multi-asset case, the correlated random vector from Cholesky decomposition is used instead of independent Z values.

5. Monte Carlo Simulation Algorithm

1. Input historical prices, asset weights, initial prices, drift, volatility, time horizon, and number of simulations.
2. Calculate historical returns and the covariance matrix.
3. Check that the covariance matrix is positive definite; add a small diagonal adjustment if required.
4. Compute the Cholesky factor L .
5. For every simulation and time step, generate a vector of independent normal values.
6. Transform the vector with L to obtain correlated shocks.
7. Update all 500 asset prices using GBM.
8. Calculate the final portfolio value and portfolio P&L;
9. Sort the simulated P&L values.
10. Calculate 99% VaR from the lower 1% tail.

6. 99% Value-at-Risk (VaR)

VaR estimates the maximum expected portfolio loss over a specified time horizon at a given confidence level. At 99% confidence, VaR corresponds to the 1st percentile of the simulated loss/P&L distribution.

If V_{init} is the initial portfolio value and V_{sim} is a simulated future value:

$$P\&L_i = V_{sim} - V_{init}$$

$$99\% \text{ VaR} = -\text{Percentile}_{99\%}(P\&L_i)$$

A positive VaR therefore represents the loss threshold that is exceeded only about 1% of the time under the simulation model.

7. Python Implementation

```
import numpy as np
```

```
# Inputs  
n_assets = 500
```

```

n_sims = 10000
days = 252

# returns: historical return matrix (observations x 500 assets)
# prices: initial prices of 500 assets
# weights: portfolio weights
mu = returns.mean(axis=0) * 252
cov = np.cov(returns, rowvar=False) * 252

# Cholesky decomposition
L = np.linalg.cholesky(cov + 1e-10 * np.eye(n_assets))

portfolio_values = np.zeros(n_sims)

for s in range(n_sims):
    prices_sim = prices.copy()

    for d in range(days):
        z = np.random.normal(size=n_assets)
        correlated_z = L @ z

        drift = (mu - 0.5 * np.diag(cov))
        diffusion = correlated_z

        prices_sim *= np.exp(
            drift / 252 + diffusion / np.sqrt(252)
        )

    portfolio_values[s] = np.sum(prices_sim * weights)

initial_value = np.sum(prices * weights)
pnl = portfolio_values - initial_value

var_99 = -np.percentile(pnl, 1)

print("Initial Portfolio Value:", initial_value)
print("99% Monte Carlo VaR:", var_99)

```

8. Expected Output

Measure	Example / Interpretation
Number of assets	500
Number of simulations	10,000
Time horizon	252 trading days (example)
Confidence level	99%
Covariance matrix	500 × 500
Portfolio VaR	Computed from the 1st percentile of simulated P&L

9. Advantages

- Captures correlations between multiple assets.
- Can model complex portfolio distributions.
- Flexible for different portfolios, time horizons, and confidence levels.
- Provides a practical framework for financial risk measurement.
- Useful for stress testing and risk management decisions.

10. Limitations

- Results depend strongly on historical data and model assumptions.
- Cholesky decomposition requires a suitable positive-definite covariance matrix.
- GBM may not fully represent market jumps, volatility clustering, or extreme events.
- Large simulations involving 500 assets can require significant computing resources.

11. Conclusion

The Dynamic Portfolio Risk Arbitrage & VaR Monte Carlo Simulator provides a systematic approach to measuring portfolio risk across 500 assets. Historical covariance captures relationships among assets, Cholesky decomposition creates correlated random shocks, and GBM generates future price scenarios. Monte Carlo simulation then produces a distribution of portfolio outcomes, from which the 99% VaR can be estimated. This approach helps analysts and risk managers understand potential downside risk and make better portfolio decisions.